

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

Trabajo Práctico N° 8

TEMA: Arreglo de Estructuras. Asignación dinámica. Pasaje de Parámetros.

¡ATENCIÓN! LEA LA CONSIGNA DE TRABAJO



- ★ Trabaje de forma ordenada en cada ejercicio, de menor a mayor.
- ★ Identifique entre la información que se brinda, el tipo de dato en cada caso y la presencia de estructuras que se puedan anidar.
- ★ Use reserva dinámica de memoria para los arreglos de tipo struct.
- ★ En los módulos, trabaje el pasaje de parámetros del tipo struct por referencia en el caso que sea necesario.

EJERCICIOS PROPUESTOS

1. Cine La Butaca

Un cine local llamado **Cine La Butaca** desea desarrollar un sistema que le permita registrar la información de las películas que se proyectan en sus salas.

El sistema debe almacenar los datos de cada película, las características principales de la sala donde se proyecta y, además, la cantidad de espectadores que asistieron a cada función.

El objetivo es organizar la información de manera que se puedan generar distintos reportes solicitados por la administración del cine.

Datos a registrar por Película

- Título
- Género (acción, comedia, drama, terror, etc.)
- Cantidad total de funciones
- Cantidad real de espectadores que asistieron a cada función
- **Sala donde se proyecta:**
 - Número de sala
 - Capacidad total de espectadores
 - Tipo de pantalla (2D, 3D, IMAX, etc.)

Informes requeridos:

1. Mostrar los datos de todas las películas cargadas.
2. Listar las películas de un género ingresado por el usuario.
3. Mostrar, para cada película cargada, el promedio de espectadores en relación a sus funciones.
4. Mostrar las películas proyectadas en salas con capacidad superior a 200 butacas.
5. Indicar cuántas películas se proyectan en pantallas de tipo "IMAX".
6. Mostrar las películas que tuvieron al menos una función con sala llena.

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

Requerimientos técnicos

- Utilice un arreglo dinámico de estructura Película.
- Para cada película, cree dinámicamente su arreglo de espectadores por función.
- La estructura Sala se encuentra anidada dentro de Película.
- Utilice pasaje por referencia en las funciones donde se modifiquen datos.
- Asegúrese de liberar toda la memoria dinámica antes de finalizar el programa.
- Inicialice los datos de forma manual o genere valores simulados para realizar las pruebas.
- Para cumplir con los informes solicitados implemente las siguientes funciones (lea atentamente cual es la tarea de cada una, tipo de retorno y argumentos):
 - **void cargarPelículas(Película *pelis, int cant);**
Carga los datos básicos de cada película (título, género, cantidad de funciones y datos de la sala). Además, debe reservar memoria para el arreglo de espectadores por función de cada película.
Los datos pueden cargarse de forma estática (asignando valores predefinidos) para facilitar las pruebas del programa.
 - **void cargarEspectadores(Película *pelis, int cant);**
Permite ingresar o generar aleatoriamente la cantidad de espectadores por función de cada película. Debe controlar que no supere la capacidad máxima de la sala.
 - **void mostrarPelículas(Película *pelis, int cant);**
Muestra los datos de todas las películas cargadas.
 - **void listarPorGenero(Película *pelis, int cant, char genero[]);**
Muestra las películas del género indicado por el usuario.
 - **float calcularPromedioEspectadores(Película p);**
Retorna el promedio de espectadores en relación a las funciones de una película específica.
 - **void mostrarSalasGrandes(Película *pelis, int cant);**
Muestra las películas proyectadas en salas con capacidad mayor a 200 butacas.
 - **int contarPelículasIMAX(Película *pelis, int cant);**
Retorna la cantidad de películas que se proyectan en salas con pantalla tipo "IMAX".
 - **void mostrarPelículasSalaLlena(Película *pelis, int cant);**
Muestra las películas que tuvieron al menos una función con sala llena (cuando la cantidad de espectadores iguala la capacidad de la sala).

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

- **void liberarMemoria(Pelicula *pelis, int cant);**

Libera toda la memoria dinámica reservada durante la ejecución del programa.

2. Realloc Soft

Una empresa de software llamada **Realloc Soft** desea implementar un sistema para gestionar sus **Proyectos** y los **Desarrolladores** asignados a cada uno.

El objetivo es llevar un registro del progreso y la composición de los equipos y los costos asociados a cada proyecto.

Datos a registrar:

- **Proyecto:**
 - Código que identifica al proyecto
 - Estado actual (ej. Análisis, Diseño, Codificación, Pruebas, Finalizado).
 - Costo total estimado.
 - Cantidad de Desarrolladores asignados.
 - Desarrolladores Asignados (arreglo dinámico)
- **Desarrollador:**
 - DNI.
 - Antigüedad en años.
 - Rol en el proyecto (ejemplo: Líder, Backend, Frontend, QA, DBA, etc.).
 - Lenguaje de programación principal (ejemplo: C, Python, Java, JavaScript, etc.).

Informes Requeridos

1. Mostrar todos los desarrolladores asignados a proyectos cuyo **estado** coincida con el ingresado por el usuario (por ejemplo, todos los desarrolladores en proyectos en "Pruebas").
2. Mostrar el **DNI** y el **rol** del desarrollador con menor antigüedad.
3. Mostrar todos los **proyectos finalizados** que involucró **desarrolladores Backend** de lenguaje **Java**.
4. Mostrar el **proyecto** que tenga la **mayor cantidad de desarrolladores** que utilicen un **lenguaje de programación específico** (ingresado por el usuario).
5. Mostrar el **costo total estimado promedio** de todos los proyectos gestionados por al menos un desarrollador con una **antigüedad superior a 5 años**.

Requerimientos técnicos

- Utilice un arreglo dinámico de estructura Proyecto.
- Para cada proyecto, cree dinámicamente su arreglo de Desarrollador.
- Trabaje con pasaje por referencia en las funciones que modifiquen datos.
- Asegúrese de liberar toda la memoria dinámica antes de finalizar el programa.

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

- Los datos pueden cargarse de forma estática o simulada, sin necesidad de ingreso por teclado.

Funciones

Utilice al menos las siguientes funciones para la implementación del programa, y agregue todas las que considere necesarias para organizar el código de forma modular y clara.

- void cargarProyectos(Proyecto *proy, int cant);
- void mostrarDesarrolladoresPorEstado(Proyecto *proy, int cant, char estado[]);
- float calcularCostoPromedioConAntiguedad(Proyecto *proy, int cant, int antiguedadMinima);
- void liberarMemoria(Proyecto *proy, int cant);

3. FleetPointers - Sistema de Gestión de Flota

Una compañía de logística llamada **FleetPointers** necesita desarrollar un sistema para administrar su flota de camiones y los mantenimientos preventivos realizados a cada uno. El sistema permitirá controlar los gastos, planificar revisiones periódicas y mantener la operatividad de los vehículos.

Datos a registrar

- **Camión**
 - Patente (código único del vehículo).
 - Marca (ejemplo: Volvo, Scania, Mercedes-Benz).
 - Año de fabricación.
 - Kilometraje actual.
 - Cantidad de mantenimientos registrados.
 - Registro de los mantenimientos.
- **Mantenimiento**
 - Fecha del mantenimiento .
 - Descripción (ejemplo: Cambio de aceite, Revisión de frenos).
 - Costo de la reparación.
 - Taller donde se realizó (nombre).
- **Fecha**
 - Día
 - Mes
 - Año

Informes y solicitudes requeridas

1. **Actualizar el kilometraje de un camión** a partir de su patente.
2. Permitir **agregar un nuevo mantenimiento** a un camión existente.
3. Mostrar el gasto total en mantenimientos de un camión específico.

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

4. Mostrar las patentes de los camiones que tienen más de 3 mantenimientos registrados.
5. Listar todos los camiones que tuvieron un costo de reparación superior a un monto ingresado por el usuario, pero solo en el mes de enero (independientemente del año).
6. Mostrar la patente y el kilometraje de los camiones cuyo año de fabricación sea anterior a 2018 y que tengan al menos un mantenimiento realizado en el taller ingresado como dato.

Requerimientos técnicos

- Utilice un arreglo dinámico de estructura Camion.
- Para cada camión, cree dinámicamente su arreglo de mantenimientos.
- La estructura Mantenimiento debe incluir una subestructura Fecha.
- Los datos pueden cargarse de forma estática o simulada, sin necesidad de ingreso por teclado.
- Utilice pasaje por referencia en las funciones que lo requieran.
- Asegúrese de liberar toda la memoria dinámica antes de finalizar el programa.
- Utilice realloc() para redimensionar el arreglo dinámico de mantenimientos al agregar un nuevo registro.

Funciones

Utilice al menos las siguientes funciones para la implementación del programa, y agregue todas las que considere necesarias para organizar el código de forma modular y clara.

- `void cargarCamiones(Camion *camiones, int cant);`
- `void actualizarKilometraje(Camion *camiones, int cant, char patente[], float nuevoKm);`
- `float calcularGastoTotal(Camion camion);`
- `void mostrarCamionesPorTaller(Camion *camiones, int cant, char taller[]);`
- Para **agregar un nuevo registro de mantenimiento** al camión, utilice una de las siguientes funciones (ambas deben usar realloc()):
 - `void agregarMantenimiento(Camion *camion);`
 - `void agregarMantenimiento(Camion *camion, Mantenimiento nuevoMant);`